

PATTERN ANALYSIS & RECOGNITION SYSTEMS LAB

Practical File Submission — Academic Session

Experiment No.: 04 | Course: PARS Lab | Framework: Python 3.14 / Flask | Repository: [dSAxmonis/pars-lab](#)

Practical Name

Bayesian Classification for Single Feature & Multiple Features Multiple Class Using Flask

AIM

To prepare an interactive web application using Flask in Python to implement Bayesian classification for both Single-Feature Multiple-Class and Multiple-Features Multiple-Class scenarios, compute prior, likelihood, and posterior probabilities, and classify test samples using the Maximum A Posteriori (MAP) decision rule.

THEORY

1. Bayesian Decision Theory

Bayesian decision theory provides a statistical framework for pattern classification by balancing prior probabilities with class-conditional likelihoods. Under Bayes' Rule:

$$P(\omega_i | \mathbf{x}) = [p(\mathbf{x} | \omega_i) \times P(\omega_i)] / p(\mathbf{x})$$
$$\text{where } p(\mathbf{x}) = \sum [p(\mathbf{x} | \omega_j) \times P(\omega_j)]$$

2. Single Feature Multiple Class (Univariate Gaussian)

For a scalar observation $x \in \mathbb{R}$, likelihood is modeled via a 1D normal distribution:

$$p(x | \omega_i) = (1 / [\sigma_i \times \sqrt{2\pi}]) \times \exp(- (x - \mu_i)^2 / (2 \sigma_i^2))$$

3. Multiple Features Multiple Class (Multivariate Gaussian)

For multidimensional vectors $\mathbf{x} \in \mathbb{R}^d$, the joint density incorporates full covariance interactions:

$$p(\mathbf{x} | \omega_i) = (1 / [(2\pi)^{(d/2)} \times |\Sigma_i|^{(1/2)}]) \times \exp(-0.5 \times (\mathbf{x} - \mu_i)^T \times \Sigma_i^{-1} \times (\mathbf{x} - \mu_i))$$

4. Maximum A Posteriori (MAP) Decision Rule

Assign $\mathbf{x}$ to class ω_k if:

$$P(\omega_k | \mathbf{x}) > P(\omega_j | \mathbf{x}) \quad \forall j \neq k$$

STEPS & EXECUTION

Step 1: Environment Setup & Dependency Verification

Created directory `exp 4/` containing `app.py` and verified dependencies (Flask, scipy, numpy, pandas, scikit-learn, matplotlib).

Step 2: Gaussian Parameter Learning via MLE

Estimated univariate and multivariate Gaussian parameters (means and covariance matrices) across all three classes.

Step 3: Bayesian Classification Inference

Evaluated likelihoods, evidence, and normalized posteriors to reach MAP classification decision.

Step 4: Interactive Web Server Launch on Port 5006

Launched interactive Flask web server on <http://127.0.0.1:5006> supporting custom test coordinates and benchmark presets.

OUTPUT / DASHBOARD RESULTS

Output 1: Learned Model Parameters & Class Priors

Displays univariate means/variances, multivariate mean vectors, and covariance matrices for Setosa, Versicolor, and Virginica.

SECTION 1: LEARNED GAUSSIAN PARAMETERS & CLASS PRIORS				
Class Label (ω_i)	Prior $P(\omega_i)$	Single Feature (Petal Length): Mean $\mu \pm \sigma$	Multi-Feature (Petal $\times$ Sepal Length): Mean Vector [μ_1, μ_2]	Covariance Matrix Σ
Setosa	0.3333	$\mu = 1.46 \text{ cm}, \sigma = 0.17 \text{ cm}$	[1.46, 5.01]	$\begin{bmatrix} 0.030 & 0.016 \\ 0.016 & 0.124 \end{bmatrix}$
Versicolor	0.3333	$\mu = 4.26 \text{ cm}, \sigma = 0.47 \text{ cm}$	[4.26, 5.94]	$\begin{bmatrix} 0.221 & 0.183 \\ 0.183 & 0.266 \end{bmatrix}$
Virginica	0.3333	$\mu = 5.55 \text{ cm}, \sigma = 0.55 \text{ cm}$	[5.55, 6.59]	$\begin{bmatrix} 0.305 & 0.303 \\ 0.303 & 0.404 \end{bmatrix}$

Figure 4.1: Section 1 — Estimated Gaussian parameters and class prior probabilities.

Output 2: Interactive Input Form & Quick Benchmarks

Interactive inputs enabling real-time classification of any custom floral measurements.

SECTION 2: INTERACTIVE TEST SAMPLE INPUT & PRESET BENCHMARKS

Feature 1 (Petal Length in cm):

Domain: [1.0 - 7.0 cm]

Feature 2 (Sepal Length in cm):

Domain: [4.0 - 8.0 cm]

Classify Test Sample (Bayesian Decision)

Quick-Select Benchmark Samples:

Benchmark Case	Petal Length	Sepal Length	Action
Preset 1: Likely Setosa	1.5 cm	5.0 cm	Test This Sample
Preset 2: Likely Versicolor	4.2 cm	5.9 cm	Test This Sample
Preset 3: Likely Virginica	5.6 cm	6.7 cm	Test This Sample
Preset 4: Borderline Versicolor/Virginica	4.9 cm	6.2 cm	Test This Sample

Figure 4.2: Section 2 — Input form and preset testing benchmark cases.

Output 3: Single-Feature & Multi-Feature Bayesian Inference

Detailed breakdown of Likelihood, Prior, Evidence, and Posterior Probabilities for Part A (Single Feature) and Part B (Multi-Feature).

SECTION 3: BAYESIAN CLASSIFICATION RESULTS & POSTERIOR PROBABILITIES							
Part A: Single Feature Multiple Class				Part B: Multiple Features Multiple Class			
Input Feature: Petal Length = 4.2 cm				Input Vector: [Petal Length: 4.2 cm, Sepal Length: 5.8 cm]			
Class (ω_i)	Likelihood $p(x \omega_i)$	Prior $P(\omega_i)$	Posterior $P(\omega_i x)$	Class (ω_i)	Likelihood $p(x \omega_i)$	Prior $P(\omega_i)$	Posterior $P(\omega_i x)$
Setosa	0.00000	0.333	0.0000 (0.0%)	Setosa	0.00000	0.333	0.0000 (0.0%)
Versicolor	0.84208	0.333	0.9590 (95.9%)	Versicolor	0.95930	0.333	0.9899 (99.0%)
Virginica	0.03597	0.333	0.0410 (4.1%)	Virginica	0.00979	0.333	0.0101 (1.0%)
Single Feature Decision: Versicolor Confidence: 95.90% (Evidence $p(x) = 0.29268$)				Multi-Feature Decision: Versicolor Confidence: 98.99% (Evidence $p(x) = 0.32303$)			

Figure 4.3: Section 3 — Side-by-side comparison of single-feature and multi-feature Bayesian decisions.

Output 4: Bayesian Decision Visualization Plots

1D Gaussian probability density curves alongside 2D scatter decision plot with test sample marker.

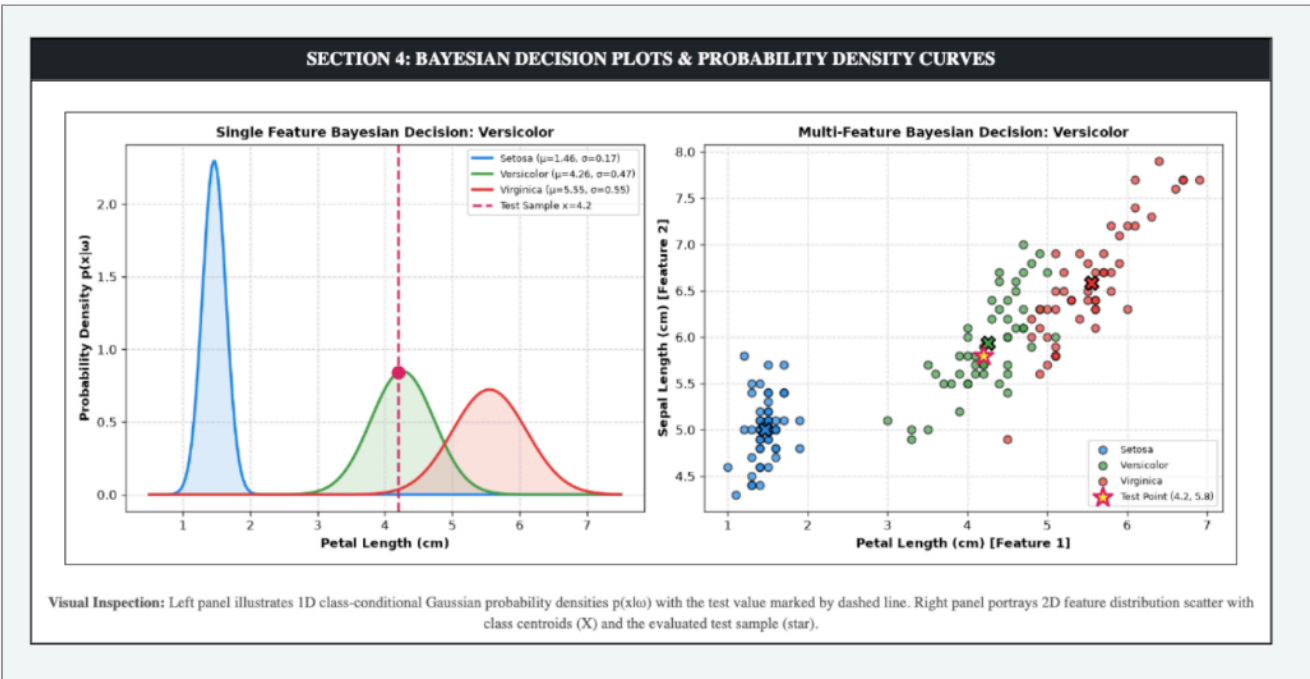


Figure 4.4: Section 4 — 1D likelihood curves and 2D feature distribution scatter plot.

CODE REPOSITORY & CLICKABLE ARTIFACT LINKS

GitHub Repository (Clickable):

GitHub: dSaxmonis/pars-lab

URL: <https://github.com/dSAxmonis/pars-lab>

Direct Source Code Files:

- **Flask Server Application:**
https://github.com/dSAxmonis/pars-lab/blob/main/exp_4/app.py
- **HTML Template:**
https://github.com/dSAxmonis/pars-lab/blob/main/exp_4/templates/index.html
- **Dependencies:**
https://github.com/dSAxmonis/pars-lab/blob/main/exp_4/requirements.txt
- **Lab Manual (Markdown):**
https://github.com/dSAxmonis/pars-lab/blob/main/exp_4/LAB_PRACTICAL_FILE.md

CONCLUSION

In this experiment, an interactive Bayesian pattern classification system was developed using Flask in Python. The implementation successfully evaluated both Single-Feature Multiple-Class (univariate Gaussian) and Multiple-Features Multiple-Class (multivariate Gaussian) architectures. Prior probabilities, class-conditional likelihoods, and posterior probabilities were computed under Bayes' rule. The Maximum A Posteriori (MAP) decision criterion was demonstrated interactively, revealing how incorporating multiple dimensions (e.g. covariance between features) sharpens class boundaries and resolves ambiguities in overlapping regions.